

INTEGRATED HOSPITAL PATIENT INFORMATION MANAGEMENT SYSTEM

B.Tech First Year – Python Programming Project

Project Type: Console-based Python Application

Storage: JSON File Storage

Academic Level: Beginner / First-Year B.Tech

Prepared according to the VITyarthi Build Your Own Project submission guidelines.

1. Introduction

The Integrated Hospital Patient Information Management System is a beginner-level Python application created to demonstrate how fundamental programming concepts can be combined to solve a practical information-management problem. The system provides separate but connected modules for patients, doctors, appointments, medical records and billing.

The VITyarthi guidelines require an original project relevant to the subject, with functional modules, clear inputs and outputs, a logical workflow, non-functional requirements, modular implementation, documentation, validation, Git usage and testing. The report and repository are structured around those requirements.

filecite turn0file0L5-L20

The Python syllabus shown in the supplied document includes fundamentals, operators, input/output, type conversion, core data structures, control flow, functions, modules/packages, arrays and object-oriented programming. These concepts are intentionally reflected in the project rather than using advanced frameworks.

2. Problem Statement

Basic hospital information can involve several related records. When these records are kept separately, finding a patient's information or connecting an appointment with a doctor can become difficult. This project proposes a simple console application that keeps these basic records in one organized system.

Problem: Create a simple, modular and easy-to-understand application that can store and retrieve basic hospital information while demonstrating first-year Python programming concepts.

3. Objectives

- Create a simple integrated hospital information system using Python.
- Provide at least three major functional modules through a single menu-driven application.
- Use input, output, conditional statements, loops and functions in a meaningful context.
- Use classes and objects to represent important entities.
- Store information persistently using JSON files.
- Validate basic user input and handle common errors.
- Provide basic testing using Python's unittest module.
- Maintain a clear GitHub-ready project structure.

4. Functional Requirements

Module	Input	Processing	Output
Patient Management	ID, name, age, gender, phone, address	Validate and save patient	Patient record / search results
Doctor Management	ID, name, department, phone	Validate and save doctor	Doctor record / search results
Appointment	Appointment ID, patient ID, doctor ID, time, date	Check IDs, time, date	Booked/cancelled appointment
Medical Records	Record ID, patient ID, doctor ID, diagnosis, prescription, notes	Check IDs, time, date	Patient medical record
Billing	Bill ID, patient ID, charges	Calculate total	Bill and total amount

The VITyarthi guideline specifies at least three major functional modules and a clear input/output structure and workflow. ■filecite■turn0file0■L24-L37■

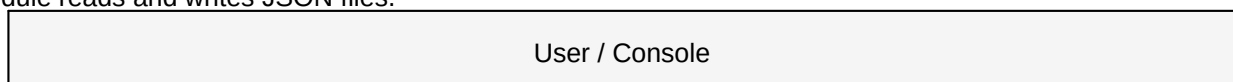
5. Non-Functional Requirements

Requirement	Project Interpretation
Usability	Simple menu-driven console interface suitable for a first-year student.
Reliability	Data is written to JSON files after successful operations.
Maintainability	Separate Python modules are used for different functional areas.
Error Handling	Input validation and handling for invalid numbers, empty values, missing records and invalid date/time for
Performance	Designed for small academic datasets and simple local operations.
Resource Efficiency	Uses standard Python libraries and small JSON files; no external database is required.

The guidelines require at least four non-functional requirements and list areas such as performance, security, usability, reliability, maintainability and error handling. ■filecite■turn0file0■L38-L50■

6. System Architecture

The project follows a simple modular architecture. The user interacts with the main menu. The main program calls the appropriate manager module. Each manager performs validation and processing, while the storage module reads and writes JSON files.



main.py – Main Menu
Patient Doctor Appointment Records Billing
storage.py – JSON File Storage
data/*.json – Local Project Data

The technical expectations include proper architecture, correct application of subject concepts, modular implementation, documentation, validation/error handling, Git and, for coding projects, 5–10 meaningful modules/classes/files with testing where applicable. ■filecite■turn0file0■L51-L63■

7. Workflow Diagram

START
Display Main Menu
Select Module
Take User Input
Validate Input
Process Data
Save / Read JSON
Display Result
Return to Main Menu / EXIT

8. Use Case Diagram

Actor	Use Cases
Hospital Operator / Student User	Register patient; add doctor; search information; book/cancel appointment; add/view medical
System	Validate input; calculate bill total; store and retrieve JSON data; display results

This tabular representation is included as a beginner-friendly equivalent of the use-case view; the final GitHub repository can also contain a visual diagram if required by the evaluator.

9. Sequence Diagram – Booking an Appointment

User	main.py	AppointmentManager	JSON Storage
1. Select Appointment	→		
2. Enter details	→	3. Validate patient/doctor	
		4. Create appointment	5. Save appointment
	←	6. Return success	←
7. Display result	←		

10. Class / Component Design

Class / Module	Main Responsibility
Patient	Represents patient information.
PatientManager	CRUD-style patient operations.
Doctor	Represents doctor information.
DoctorManager	Doctor registration and search.
Appointment	Represents an appointment.
AppointmentManager	Booking, viewing and cancellation.
MedicalRecord	Represents a patient record.
RecordManager	Adding and viewing medical records.
Bill	Calculates and stores bill total.
BillingManager	Creating and viewing bills.
Storage functions	Read/write JSON data.

11. ER / Data Storage Design

Entity	Important Fields	Relationship
Patient	patient_id, name, age, gender, phone, address	One patient can have multiple appointments, records and bills.
Doctor	doctor_id, name, department, phone	One doctor can be linked to multiple appointments and records.
Appointment	appointment_id, patient_id, doctor_id, date, time, reason, status	Connects a patient with a doctor.
Medical Record	record_id, patient_id, doctor_id, diagnosis, prescription, notes	Belongs to a patient and references a doctor.
Bill	bill_id, patient_id, charges, total, status	Belongs to a patient.

12. Design Decisions & Rationale

- **Console interface:** chosen because it is achievable and understandable for a first-year Python project.
- **JSON storage:** avoids database setup while still demonstrating file handling and persistence.
- **Multiple modules:** separates responsibilities and demonstrates Python modules and clean organization.
- **Classes:** used for Patient, Doctor, Appointment, MedicalRecord and Bill to introduce OOP concepts.
- **Simple validation:** prevents empty inputs, duplicate IDs and references to missing patients/doctors.
- **No advanced framework:** keeps the project aligned with the beginner syllabus and avoids hiding basic programming concepts.

13. Implementation Details

- main.py controls the menu and connects the modules.
- patient.py manages patient records and update/search operations.
- doctor.py manages doctor records.
- appointment.py validates patient/doctor IDs and date/time before booking.
- records.py stores diagnosis, prescription and notes.
- billing.py calculates total charges using arithmetic operations.
- storage.py provides reusable JSON load/save functions.
- utils.py provides reusable input validation and menu utilities.
- test_system.py checks basic object creation, appointment status and bill calculation.

- The project contains more than the required five meaningful Python files/modules.

14. Screenshots / Results

Example successful workflow

1. User selects Patient Management → Register Patient.
2. Example input: P001, Asha, 20, Female, 9999999999, Bhopal.
3. System displays: Patient registered successfully.
4. User adds doctor D001 and books appointment A001.
5. User adds medical record R001 and creates bill B001.
6. System displays the stored records when requested.

Example bill result: Consultation ■500 + Tests ■300 + Medicines ■200 = Total ■1000.

Automated test result:

Spreadsheet runtime warmup failed during python startup

Traceback (most recent call last):

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_on_startup

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 720, in _warm_feature_flows

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 704, in _warm_collaboration_flows

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/generated/interface/models.py", line 32317, in hydrate_crdt_from_proto

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/remote.py", line 749, in __call__

File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/client.py", line 150, in call
artifact_tool.rpc.client.RemoteError: hydrateCrdtFromProto requires an empty collaborative document.

....

Ran 4 tests in 0.000s

OK

15. Testing Approach

Test	Expected Result	Status
Create Patient object	Correct ID and age stored	Passed
Create Doctor object	Department stored	Passed
Create Appointment	Default status is Booked	Passed
Create Bill	Total equals sum of charges	Passed
Empty required input	User is asked again	Implemented
Duplicate IDs	Operation is rejected	Implemented
Invalid age/number	Validation message shown	Implemented
Invalid appointment date/time	Booking is rejected	Implemented

16. Challenges Faced

- Connecting records across different modules using IDs.
- Handling invalid input without crashing the program.
- Keeping code separated into manageable modules.
- Saving and loading JSON data correctly.

- Designing a project that is useful while remaining appropriate for first-year programming skills.

17. Learnings & Key Takeaways

- Learned how a real-world problem can be converted into programming modules.
- Practiced functions, loops, conditions and input/output.
- Applied lists and dictionaries to organize information.
- Practiced classes, constructors, methods and basic object-oriented design.
- Learned basic file handling and JSON persistence.
- Learned the importance of validation, testing and modular code.
- Understood the role of GitHub in organizing and submitting a coding project.

18. Future Enhancements

- Add a graphical user interface.
- Replace JSON files with a relational database such as SQLite.
- Add login and role-based access.
- Add stronger privacy and security controls.
- Add reporting/dashboard features.
- Add appointment availability checking.
- Add automated backups and audit logs.
- Add professional deployment and security review before any real-world use.

19. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (supplied PDF).
- Python documentation and standard library concepts used in the project.
- Course topics shown in the supplied Python module screenshots.

20. Submission Checklist

Required Item	Included in Package
README.md	Yes
statement.md	Yes
Source code / project files	Yes
5–10 meaningful modules/classes/files	Yes – modular Python files included
Data files	Yes – JSON files
Testing	Yes – test_system.py
Project report PDF	Yes
Design artefacts	Yes – architecture, workflow, use-case, sequence, class/component and storage diagrams
Git version control	Ready for GitHub upload; Git history must be created by the student

The official guidelines specify README.md, statement.md and organized source files in the GitHub repository, and a separate detailed PDF report for portal submission. ■filecite■turn0file0■L87-L106■

The report sections above follow the required report sequence, including cover page, introduction, problem statement, requirements, architecture/design diagrams, implementation, results, testing, challenges, learnings, future enhancements and references. ■filecite■turn0file0■L109-L134■